

Design and Implementation of an Optimal Binary Search Tree using Dynamic Programming

WALID M. W. KOBHALABI

Department of Artificial Intelligence Engineering

Faculty of Engineering and Natural Sciences

Bahçeşehir University, İstanbul, Türkiye

walidmw.kobahalabi@bahcesehir.edu.tr

CMP3005 – Analysis of Algorithms

Supervised by Dr. Md Imran Hosen

Abstract— A binary search tree may be arranged in many different shapes over the same set of keys, and when some keys are searched more frequently than others the chosen arrangement determines how many comparisons an average search requires. This project presents the design, implementation and experimental analysis of an Optimal Binary Search Tree (OBST) constructed by dynamic programming. The problem is formulated as a set of overlapping subproblems in which the state $C[i, j]$ denotes the minimum expected search cost over the contiguous key range $k_i \dots k_j$, and the recurrence evaluates every candidate root within that range. The implementation, written in Python, builds the cost table and the root table, reconstructs the optimal tree from the root table, reports the depth of every key, and independently recomputes the expected cost by traversing the reconstructed tree in order to verify the result. On the five-key example specified in the project description the algorithm produces an expected search cost of 1.80 comparisons, against 1.90 for a height-balanced tree and 3.00 for a tree built by sequential insertion of sorted keys. Execution time was measured for seven input sizes between $n = 5$ and $n = 400$. The measured cost per inner-loop operation converges towards a constant of approximately $0.05 \mu s$, and doubling n from 200 to 400 multiplied the running time by a factor of 8.9 against the factor of 8 predicted by cubic growth, confirming the theoretical complexity of $O(n^3)$ time and $O(n^2)$ space.

Index Terms— Optimal Binary Search Tree (OBST), Dynamic Programming (DP), Expected Search Cost, Overlapping Subproblems, Optimal Substructure, Complexity Analysis.

I. INTRODUCTION

The binary search tree is one of the fundamental data structures used for storing an ordered collection of keys. Searching for a key in such a tree requires one comparison at every node visited on the path from the root, so the cost of a single search is equal to the depth of the key being sought. When all keys are equally likely to be requested, the natural objective is to keep the tree as shallow as possible, and a height-balanced tree achieves this.

In many practical situations, however, the keys are not requested with equal frequency. A compiler symbol table, a dictionary of natural language words and a database index are all examples in which a small number of entries account for a large proportion of the accesses. Under these conditions minimizing the height of the tree is no longer the correct objective.

What should be minimized instead is the *expected* number of comparisons, which weights the depth of each key by the probability that the key is requested. A tree that places a frequently requested key close to the root may outperform a perfectly balanced tree even though it is taller, and the advantage grows as the access distribution becomes more skewed. This is the Optimal Binary Search Tree problem, stated in the course material as the task of finding, for n keys $a_1 < \dots < a_n$ with search probabilities $p_1, \dots, p_n$, the binary search tree with the minimum average number of comparisons in a successful search [2]. The difficulty is that the number of distinct binary search trees over n keys grows exponentially, so brute force is not a usable strategy. For $n = 5$ there are 42 possible trees, and for $n = 20$ there are approximately 6.56×10^9 .

Dynamic programming solves the problem efficiently because it is a technique for problems that can be formulated as recurrences with overlapping sub-instances, where smaller instances are solved once, recorded in a table, and the solution to the original instance is then extracted from that table [2]. The algorithm implemented in this project solves the case $n = 20$ using 1,540 inner-loop operations rather than examining billions of candidate trees.

The Optimal Binary Search Tree belongs to a wider family of problems solved by this technique, alongside the longest common subsequence, Warshall's algorithm for transitive closure, Floyd's algorithm for all-pairs shortest paths and the knapsack problem [2]. In common with several of these, it requires more than the value of the optimum: the structure achieving that value must also be recovered, which is done here by recording the chosen root of every subproblem in a second table and reading it back afterwards.

The report is organised as follows. Sections II to IV define the problem and derive the formulation and the algorithm. Sections V to VIII cover the implementation, the experimental results, the comparison against conventional trees and the complexity analysis. Sections IX and X discuss the findings and conclude.

Source code GitHub repository:

<https://github.com/WalidMWH/optimal-bst-dynamic-programming>

II. PROBLEM DEFINITION

A. Formal Statement

Let $K = \{k_1, k_2, \dots, k_n\}$ be a set of keys given in sorted order, so that $k_1 < k_2 < \dots < k_n$. Let p_i denote the probability that a search is made for key k_i . These are successful-search probabilities only; every search is assumed to find its key, and therefore:

$$\sum_{i=1}^n p_i = 1, \quad p_i \geq 0. \quad (1)$$

For a given binary search tree T over these keys, let $\text{depth}_T(k_i)$ be the depth at which key k_i is stored, with the root of the tree taken to be at depth 1. Since locating a key at depth d requires exactly d comparisons, the expected number of comparisons per search is:

$$E(T) = \sum_{i=1}^n p_i \cdot \text{depth}_T(k_i). \quad (2)$$

The problem is to find the tree T^* that minimises $E(T)$ over all binary search trees on K [1].

B. Why the Arrangement of Keys Affects the Cost

The probabilities in (2) are fixed by the input and cannot be altered. The only quantities under the control of the algorithm are the depths. Every valid binary search tree over the same sorted keys contains exactly the same keys and satisfies the same ordering property, so the trees differ only in the assignment of depths to keys. Minimizing the expected cost is therefore equivalent to finding the assignment of depths that pairs the largest probabilities with the smallest depths, subject to the constraint that the resulting assignment must correspond to an actual binary search tree.

It is instructive to see that a greedy strategy does not solve this problem. Placing the highest-probability key at the root appears attractive, but that single choice fixes which keys must lie in the left sub-tree and which in the right, and a locally favorable choice may force an unfavorable arrangement below it. Section IX returns to this point using the ten-key instance of Section VI, in which the key with the second-highest probability is placed at depth 3 while a key with less than half its probability is placed at depth 2.

C. The Cost of Exhaustive Search

Since a brute-force approach is conceptually simple, it is worth establishing precisely why it is unusable. The total number of binary search trees with n nodes is $C(2n, n)/(n+1)$, which grows exponentially, so brute force is hopeless [2].

This quantity is the Catalan number:

$$C_n = \frac{1}{n+1} \binom{2n}{n}. \quad (3)$$

Table I compares it against the number of elementary operations performed by the dynamic programming algorithm developed in Section III. The gap becomes decisive well before n reaches the sizes tested in this project.

TABLE I
NUMBER OF CANDIDATE TREES AGAINST DP OPERATIONS

n	Distinct BSTs C_n	DP operations
5	42	35
10	16,796	220
20	6.56×10^9	1,540
50	1.98×10^{27}	22,100
100	8.96×10^{56}	171,700

D. A Worked Example by Exhaustive Enumeration

To confirm that the optimum found later by dynamic programming is genuinely the global minimum, consider the small instance $K = \{10, 20, 30\}$ with probabilities $p = \{0.2, 0.5, 0.3\}$. Since $C_3 = 5$, all five trees can be enumerated by hand. Table II lists them with the depth of each key and the resulting expected cost computed from (2).

TABLE II
ALL FIVE BINARY SEARCH TREES ON THREE KEYS

#	Root arrangement	Depths			$E(T)$
		10	20	30	
1	Right chain from 10	1	2	3	2.10
2	10 root, 30 above 20	1	3	2	2.30
3	20 at root	2	1	2	1.50
4	30 root, 10 above 20	2	3	1	2.20
5	Left chain from 30	3	2	1	1.90

The minimum is 1.50, achieved by the tree rooted at key 20. This value is reproduced by the dynamic programming algorithm in Section III, providing an independent confirmation of correctness.

III. DYNAMIC PROGRAMMING FORMULATION

A. Definition of the State

Following the formulation given in the project description [1] and the course material [2], [3], the dynamic programming state is defined as:

$$C[i, j] = \text{minimum expected search cost of OBST over keys } k_i \dots k_j. \quad (4)$$

Where the depth of each key is measured relative to the root of that sub-tree, not relative to the root of the whole tree. The relative measurement of depth is essential and is what makes the dynamic programming approach valid. If depths were measured from the global root, the cost of a sub-tree would depend on where that sub-tree was eventually attached, and the subproblems would no longer be independent of one another. By measuring depth locally, the value $C[i, j]$ becomes a property of the key range alone and may be computed once and reused wherever that range appears.

The final answer to the problem is $C[1, n]$.

B. Optimal Substructure

Suppose the optimal tree over the range $k_i \dots k_j$ has key k_r at its root, for some $i \leq r \leq j$. The binary search tree property then forces the keys $k_i \dots k_{r-1}$ into the left sub-tree and the keys $k_{r+1} \dots k_j$ into the right sub-tree; no other partition is possible.

Furthermore, both of those sub-trees must themselves be optimal for their respective ranges. If the left sub-tree could be replaced by a cheaper tree over the same keys, substituting it would reduce the cost of the whole tree, contradicting the assumption that the tree was optimal. The problem therefore exhibits optimal substructure, which is the first condition required for dynamic programming to apply.

C. Derivation of the Recurrence

Consider choosing k_r as the root of the range $k_i \dots k_j$. The two sub-trees are optimal solutions to smaller subproblems, contributing $C[i, r-1]$ and $C[r+1, j]$ respectively. However, these two values were computed with depths measured relative to the roots of the sub-trees themselves.

When the sub-trees are attached beneath k_r , every key inside them moves down by exactly one level, and each therefore requires one additional comparison. The key k_r itself, now at depth 1 of the combined tree, requires one comparison. Every key in the range $k_i \dots k_j$ thus contributes exactly one extra comparison relative to its position within its own sub-tree.

The total additional cost is the sum of the probabilities of all keys in the range, denoted:

$$W(i, j) = \sum_{s=i}^j p_s. \quad (5)$$

Because the optimal root is not known in advance, every candidate must be tried and the cheapest retained.

This yields the recurrence:

$$C[i, j] = \min_{i \leq r \leq j} \{C[i, r-1] + C[r+1, j]\} + W(i, j) \quad (6)$$

for $1 \leq i \leq j \leq n$.

An important observation is that $W(i, j)$ does not depend on r . It is therefore constant across the entire minimization and may be added once after the minimum has been found, rather than inside the loop over candidate roots.

D. Base Cases

The only base case required is that of the empty range:

$$C[i, j] = 0 \quad \text{when } i > j. \quad (7)$$

An empty sub-tree contains no keys, so it contributes nothing to the expected cost. The single-key case stated in the project description, $C[i, i] = p_i$ [1], does not need to be given separately because it follows from equations (6) and (7).

Setting $i = j$ leaves only one possible root, $r = i$, and substitution gives:

$$\begin{aligned} C[i, i] &= C[i, i-1] + C[i+1, i] + W(i, i) \\ &= 0 + 0 + p_i = p_i. \end{aligned} \quad (8)$$

Both sub-ranges are empty and therefore contribute zero, and the interval weight of a single key is its own probability. The value p_i corresponds to that key sitting alone at depth 1, which is consistent with the convention adopted throughout this report.

E. Overlapping Subproblems

The recurrence (6) refers to subproblems of the form of $[i, r-1]$ and $[r+1, j]$. A given range such as $[2, 4]$ is required when solving $[1, 4]$ with root k_1 , when solving $[2, 5]$ with root k_5 , and in several other contexts. A naive recursive evaluation would recompute these values repeatedly, giving exponential running time.

Since there are only $\Theta(n^2)$ distinct ranges, storing each result in a table and reusing it eliminates all recomputation. This is the second condition required for dynamic programming, and it is what reduces the running time from exponential to polynomial.

F. Worked Example

The three-key instance introduced in Section II is now solved using the recurrence where the prefix sums of the probabilities $p = \{0.2, 0.5, 0.3\}$ are $S = [0, 0.2, 0.7, 1.0]$, from which any $W(i, j)$ is obtained as $S_j - S_{i-1}$.

- **Ranges of length 1:** Following directly from (8), the costs are:

$$C[1, 1] = 0.2$$

$$C[2, 2] = 0.5$$

$$C[3, 3] = 0.3$$

- **Ranges of length 2:** For the range $[1, 2]$, evaluating the two candidate roots yields:

$$r = 1 : C[1, 0] + C[2, 2] = 0 + 0.5 = 0.5$$

$$r = 2 : C[1, 1] + C[3, 2] = 0.2 + 0 = 0.2$$

The minimum is 0.2, and adding $W(1, 2) = 0.7$ gives $C[1, 2] = 0.9$ with root index 2. Similarly, computations for the range $[2, 3]$ give $C[2, 3] = 1.1$ with root index 2.

- **Ranges of length 3:** For the full range $[1, 3]$, evaluating the three candidate roots yields:

$$r = 1 : C[1, 0] + C[2, 3] = 0 + 1.1 = 1.1$$

$$r = 2 : C[1, 1] + C[3, 3] = 0.2 + 0.3 = 0.5$$

$$r = 3 : C[1, 2] + C[4, 3] = 0.9 + 0 = 0.9$$

The minimum is 0.5, and adding $W(1, 3) = 1.0$ gives the final expected cost:

$$C[1, 3] = 0.5 + 1.0 = 1.50 \quad (9)$$

with root index 2, corresponding to key 20. This agrees exactly with the exhaustive enumeration in Table II, confirming that the recurrence identifies the true global optimum.

IV. ALGORITHM AND PSEUDOCODE

A. Order of Evaluation

The recurrence (6) computes $C[i, j]$ from values over strictly shorter ranges. The table must therefore be filled in increasing order of range length, so that every value required is already final when it is read. Iterating over i and j independently would not guarantee this ordering and would read uninitialized entries.

The tables are consequently filled diagonal by diagonal [2]: all ranges of length 1 are solved first, then all of length 2, and so on until the single range of length n is reached.

B. Efficient Evaluation of the Interval Weight

The term $W(i, j)$ is required once for every range, of which there are $\Theta(n^2)$. Evaluating it by summing p_i through p_j each time would cost $O(n)$ per range and would raise the overall running time to $O(n^4)$. This is avoided by pre-computing the prefix sums:

$$S_0 = 0, \quad S_j = \sum_{s=1}^j p_s, \quad (10)$$

in a single linear pass, after which any interval weight is obtained in constant time as:

$$W(i, j) = S_j - S_{i-1}. \quad (11)$$

C. Construction of the Tables

Algorithm 1 constructs both the cost table and the root table. The root table records, for every range, which key was selected as its root; it carries no cost information but is what allows the tree itself to be rebuilt afterwards.

Algorithm 1 Optimal BST table construction

Require: Probabilities $p_1 \dots p_n$
Ensure: Cost table C and root table R

```

1:  $S \leftarrow \text{PREFIXSUMS}(p)$ 
2: initialise  $C[i][j] \leftarrow 0$  and  $R[i][j] \leftarrow 0$  for all  $i, j$ 
3: for  $length \leftarrow 1$  to  $n$  do
4:   for  $i \leftarrow 1$  to  $n - length + 1$  do
5:      $j \leftarrow i + length - 1$ 
6:      $w \leftarrow S[j] - S[i - 1]$ 
7:      $best \leftarrow \infty$ ;  $root \leftarrow i$ 
8:     for  $r \leftarrow i$  to  $j$  do
9:        $t \leftarrow C[i][r - 1] + C[r + 1][j]$ 
10:      if  $t < best$  then
11:         $best \leftarrow t$ ;  $root \leftarrow r$ 
12:      end if
13:    end for
14:     $C[i][j] \leftarrow best + w$ 
15:     $R[i][j] \leftarrow root$ 
16:  end for
17: end for
18: return  $C, R$ 
```

Two details of Algorithm 1 deserve comment. The candidate roots are examined in ascending order and a new best is accepted only on a strict improvement, so that when several roots achieve the same minimum the smallest index is retained. This makes the output deterministic and reproducible.

In addition, the tables are allocated with two extra rows and columns rather than one, because line 9 reads index $r - 1$,

which is 0 when $r = i = 1$, and index $r + 1$, which is $n + 1$ when $r = j = n$.

Those padding entries hold the value zero and represent the empty sub-trees of (7), so the base case is encoded structurally and requires no conditional test inside the innermost loop.

D. Reconstruction of the Tree

The cost table alone gives only the value of the optimum. The tree itself is rebuilt from the root table by the recursive procedure of Algorithm 2, which reads the stored root of a range, creates a node for it, and recurses on the two sub-ranges one level deeper.

Algorithm 2 Reconstruction from the root table

```

1: function BUILDTREE( $k, p, R, i, j, d$ )
2:   if  $i > j$  then
3:     return NULL
4:   end if
5:    $r \leftarrow R[i][j]$ 
6:    $v \leftarrow$  new node with key  $k_r$ , probability  $p_r$ , depth  $d$ 
7:    $v.left \leftarrow$  BUILDTREE( $k, p, R, i, r - 1, d + 1$ )
8:    $v.right \leftarrow$  BUILDTREE( $k, p, R, r + 1, j, d + 1$ )
9:   return  $v$ 
10: end function
```

The procedure is invoked as BUILDTREE($k, p, R, 1, n, 1$), so that the root of the complete tree receives depth 1.

E. Verification of the Result

The project description requires that the expected cost be verified using the constructed tree [1]. This is achieved by traversing the reconstructed tree and accumulating $p_i \cdot \text{depth}(k_i)$ over all nodes, as given directly by the definition in (2).

This computation never consults the cost table. If the value obtained agrees with $C[1, n]$, then two independent computations following different routes have produced the same result, which confirms that the reconstruction correctly realises the optimum found by the dynamic program.

V. IMPLEMENTATION

A. Structure of the Program

The program is written in Python 3.13 and is organized into four modules under a `src` directory, together with a command-line entry point and a separate experimental harness. The division of responsibilities is summarized in Table III.

TABLE III
PROGRAM MODULES AND THEIR RESPONSIBILITIES

Module	Responsibility
<code>src/obst.py</code>	Prefix sums, cost table, root table
<code>src/tree.py</code>	Reconstruction, depths, verification
<code>src/baseline.py</code>	Conventional BSTs for comparison
<code>src/validation.py</code>	Input reading and validation
<code>main.py</code>	Command-line interface and reporting
<code>experiments.py</code>	Timing across input sizes
<code>tests/</code>	Automated correctness tests

The complete source code is available in the project repository [4]. No third-party library is required by the core algorithm; only the Python standard library is used. The plotting package `matplotlib` is imported inside the plotting function and guarded by an exception handler, so the program remains fully usable when it is not installed.

B. Core Implementation

Listing 1 shows the function that constructs both tables. It corresponds directly to Algorithm 1.

Listing 1 Construction of the cost and root tables

```

1 def build_dp_tables(probabilities):
2     n = len(probabilities) - 1
3     prefix = compute_prefix_sums(probabilities)
4
5     # Empty intervals remain 0.0
6     cost = [[0.0] * (n + 2) for _ in range(n + 2)]
7     root = [[0] * (n + 2) for _ in range(n + 2)]
8
9     # Shortest intervals are solved first
10    for length in range(1, n + 1):
11        for i in range(1, n - length + 2):
12            j = i + length - 1
13
14            # Constant time interval weight
15            weight = interval_probability(prefix, i, j)
16            best_cost = float("inf")
17            best_root = i
18
19            # Try every possible root in the range
20            for r in range(i, j + 1):
21                subtree_cost = cost[i][r-1] + cost[r+1][j]
22                if subtree_cost < best_cost:
23                    best_cost = subtree_cost
24                    best_root = r
25
26            cost[i][j] = best_cost + weight
27            root[i][j] = best_root
28
29    return cost, root

```

C. Implementation Decisions

Several decisions were fixed at the outset so that the code, the mathematics and the output would remain consistent with one another.

- **Indexing:** All arrays are one-based, with index 0 present as unused padding. This allows the code to use exactly the same indices as the recurrence in Section III, which removes a common source of off-by-one errors and makes the implementation directly comparable with the formulation.
- **Depth convention:** The root of a tree is at depth 1. This follows from the base case derivation in (8) and gives the expected cost the natural interpretation of an expected number of comparisons.
- **Tie-breaking:** When several candidate roots achieve the same minimum, the smallest index is selected. The choice is arbitrary in terms of cost, since all tied roots produce trees of equal expected cost, but fixing it makes the output deterministic across runs.
- **Formulation:** The formulation used is the one given in the project description and the course material, using successful-search probabilities only [1], [2]. The more general treatment, in which the method is expanded to include unsuccessful searches, is discussed in Section IX as a possible extension but is not implemented here.

D. Input Handling and Validation

The program accepts input from a file, from an interactive prompt, or from a seeded random generator used by the experiments.

In all cases the input is validated before the dynamic program is invoked, as required by the project description [1]. The checks are applied in a fixed order: list lengths must agree, at least one key must be present, keys must be distinct and strictly increasing, no probability may be negative, and the probabilities must sum to 1 within a tolerance of 10^{-6} .

The ordering of these checks is significant rather than arbitrary. Two of the invalid test files supplied with the project have probabilities that sum to exactly 1.0 and would pass the summation test; they are rejected only because the ordering check and the negativity check are performed first. Each failure raises a single descriptive error naming the specific fault, and the program exits with a non-zero status without printing a stack trace.

E. Automated Testing

To ensure reliability, the implementation is supported by a suite of 185 automated tests, structured into three primary categories:

- 1) **Hand-computed validation:** Pins the manual results of Section III in place by verifying every individual entry of both the cost and root tables for the 3-key and 5-key instances, ensuring accuracy throughout the entire matrix rather than just checking the final answer.
- 2) **Invariant checking:** Verifies structural properties using randomly generated instances up to $n = 30$. These tests confirm that the actual tree traversal cost matches $C[1, n]$, an in-order traversal exactly reproduces the sorted input keys without duplicates, and the optimal cost never exceeds that of conventional baselines.
- 3) **Exhaustive cross-checking:** Compares the dynamic program against a brute-force enumeration of all binary search trees for inputs up to $n = 7$ (evaluating all 429 possible tree shapes). While exponential scaling limits this to small inputs, it provides a complete mathematical proof of correctness for those sizes rather than a simple spot check.

VI. EXPERIMENTAL RESULTS

A. Results for the Specified Example

The example given in the project description consists of five keys with the probabilities shown in Table IV [1].

TABLE IV
INPUT SPECIFIED IN THE PROJECT DESCRIPTION

Index	Key	Probability
1	10	0.10
2	20	0.20
3	30	0.40
4	40	0.20
5	50	0.10
Sum		1.00

The cost table produced by the program is shown in Table V. Only the upper triangle is used, since $C[i, j]$ is defined only for $i \leq j$. The minimum expected search cost for the complete problem is the top-right entry, $C[1, 5] = 1.80$.

TABLE V
DYNAMIC PROGRAMMING COST TABLE $C[i, j]$

$i \setminus j$	1	2	3	4	5
1	0.10	0.40	1.10	1.50	1.80
2		0.20	0.80	1.20	1.50
3			0.40	0.80	1.10
4				0.20	0.40
5					0.10

The corresponding root table is given in the Table VI.

Where each entry records the index of the key chosen as the root of that range.

TABLE VI
ROOT TABLE RECORDING THE SELECTED ROOT OF EACH SUBPROBLEM

$i \setminus j$	1	2	3	4	5
1	1	2	3	3	3
2		2	3	3	3
3			3	3	3
4				4	4
5					5

B. The Reconstructed Tree

Reading $R[1, 5] = 3$ selects key 30 as the root. This splits the problem into the ranges $[1, 2]$ and $[4, 5]$, whose stored roots are keys 20 and 40 respectively, and the recursion continues until all keys are placed. The resulting tree is shown in Fig. 1.

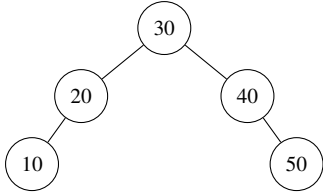


Fig. 1. The optimal binary search tree, expected cost 1.80

C. Verification of the Expected Cost

Table VII shows the depth assigned to each key together with its contribution to the expected cost.

TABLE VII
DEPTH OF EACH KEY AND VERIFICATION OF THE EXPECTED COST

Key	Probability	Depth	$p_i \times \text{depth}$
10	0.10	3	0.30
20	0.20	2	0.40
30	0.40	1	0.40
40	0.20	2	0.40
50	0.10	3	0.30
Total			1.80

The total obtained by traversing the reconstructed tree is 1.80, which agrees with the value $C[1, 5]$ read from the cost table. The absolute difference reported by the program is 2.22×10^{-16} , which is the resolution of double-precision floating-point arithmetic and reflects the order in which the additions were performed rather than any disagreement between the two methods.

D. Validation Against the Course Example

As an additional check, the implementation was run on the worked example given in the course material, which uses four keys A, B, C, D with probabilities 0.1, 0.2, 0.4 and 0.3 respectively [2]. The keys were encoded as the integers 1 to 4 to preserve their order.

The program returns a minimum expected cost of 1.70 with key C at the root, B and D at depth 2 and A at depth 3. Both the cost and the tree structure match the solution given in the course material exactly, and the computed cost and root tables reproduce the tables presented there entry for entry. This confirms that the implementation agrees with an independently published solution and not only with the author's own hand calculations.

E. Results for a Ten-Key Instance

The five-key example is small enough that its optimal tree is symmetric and somewhat unrepresentative. A second instance was therefore constructed with ten keys, 10 through 100, carrying the probabilities listed in Table VIII.

The distribution peaks at key 50 with probability 0.20 and at key 60 with probability 0.18, and tapers towards both ends.

TABLE VIII
TEN-KEY INSTANCE: PROBABILITIES, DEPTHS AND VERIFICATION

Key	Probability	Depth	$p_i \times \text{depth}$
10	0.04	4	0.16
20	0.06	3	0.18
30	0.08	2	0.16
40	0.10	3	0.30
50	0.20	1	0.20
60	0.18	3	0.54
70	0.12	2	0.24
80	0.10	4	0.40
90	0.08	3	0.24
100	0.04	4	0.16
Total			2.58

TABLE IX
ROOT TABLE FOR THE TEN-KEY INSTANCE, SHOWING THE MONOTONICITY OF THE SELECTED ROOTS

$i \setminus j$	1	2	3	4	5	6	7	8	9	10
1	1	2	2	3	4	5	5	5	5	5
2		2	3	3	4	5	5	5	5	5
3			3	4	5	5	5	5	6	6
4				4	5	5	5	6	6	6
5					5	5	6	6	6	6
6						6	6	7	7	7
7							7	7	8	8
8								8	8	9
9									9	9
10										10

The dynamic program returns a minimum expected search cost of $C[1, 10] = 2.58$, with $R[1, 10] = 5$ selecting key 50 as the root. The full root table is given in Table IX and the reconstructed tree in Fig. 2.

Summing the final column of Table VIII reproduces 2.58 exactly; for this instance the absolute difference between the two computations is zero rather than merely small, since the probabilities happen to be exactly representable in binary floating point.

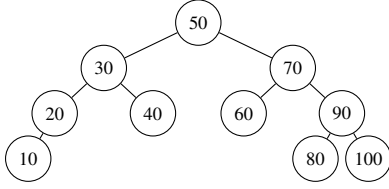


Fig. 2. Optimal tree for the ten-key instance, expected cost 2.58

Two features of this result are worth noting immediately. The tree is asymmetric, unlike the five-key case. More significantly, key 60 carries the second-largest probability in the input yet is placed at depth 3, below key 30 at depth 2 whose probability is less than half as large. This apparent inversion is examined in Section IX.

F. Execution Time Across Input Sizes

The running time of the table construction was measured for the input sizes recommended in the project description, extended with two larger sizes [1]. Each measurement is the mean of five independent runs on randomly generated probability distributions, timed with the Python `time.perf_counter()` function. All measurements were taken on a Windows laptop running Python 3.13. The extension to $n = 200$ and $n = 400$ was made because at the sizes originally specified the running time is short enough that fixed interpreter overhead forms a substantial fraction of each measurement, which obscures the underlying growth rate. The results appear in Table X.

TABLE X
EXPERIMENTAL RESULTS FOR EACH INPUT SIZE

Keys n	Time (s)	Min. cost	Remarks
5	0.000008	1.8938	Overhead dominated
10	0.000024	2.4129	Overhead dominated
20	0.000113	3.1465	Overhead share falling
50	0.001168	4.3099	First result above 1 ms
100	0.008437	5.2205	Largest required size
200	0.061855	6.1840	Cubic growth visible
400	0.550207	7.1608	Time $\times 8.9$ for $n \times 2$

TABLE XI
OPERATION COUNT AND COST PER OPERATION

Keys n	Inner-loop operations	Time per operation (μ s)
5	35	0.243
10	220	0.109
20	1,540	0.074
50	22,100	0.053
100	171,700	0.049
200	1,353,400	0.046
400	10,746,800	0.051

The minimum expected cost rises with n because a larger key set produces a deeper tree; each figure is the mean over the five random instances used at that size. Table XI gives the exact number of candidate roots evaluated by the algorithm, derived in Section VIII as $n(n+1)(n+2)/6$.

G. Relating Time to Operation Count

The third column of Table XI divides the measured time by the operation count, giving the average cost of a single inner-loop iteration. If the implementation is genuinely cubic, this quantity should approach a constant as n grows, because the total time is then simply the operation count multiplied by a fixed per-operation cost.

The values fall steeply at first and then settle at approximately 0.05μ s, remaining nearly constant across the last three sizes despite the operation count increasing by a factor of more than sixty over that range. This convergence is direct empirical evidence of cubic behaviour.

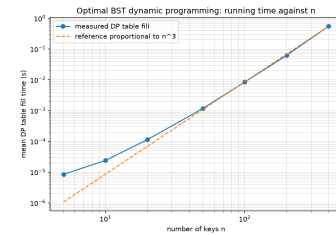
A second view of the same data is obtained by examining growth ratios between consecutive sizes. Doubling n should multiply a cubic running time by $2^3 = 8$. From $n = 100$ to $n = 200$ the measured time increased by a factor of 7.33, and from $n = 200$ to $n = 400$ by a factor of 8.90, bracketing the predicted value.

H. Empirical Growth Exponent

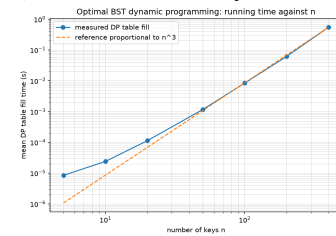
Fitting a straight line by least squares through $\log n$ against $\log t$ estimates the exponent of the growth. Across all seven sizes the fitted exponent is 2.565. Restricting the fit to $n \geq 20$ raises it to 2.836.

The difference between the two figures is itself informative. At $n = 5$ the algorithm performs only 35 operations, while the cost of entering the function, allocating the tables and reading the clock is roughly independent of n . That fixed cost inflates the smallest measurements relative to the cubic trend and flattens the fitted line. As the smallest sizes are excluded, the estimate rises towards the theoretical value of 3.

Fig. 3 presents both fits on logarithmic axes together with a reference line proportional to n^3 , anchored in each case at the largest measured point so that only the slopes may be compared.



(a) All seven sizes; fitted exponent 2.565.



(b) Sizes $n \geq 20$ only; fitted exponent 2.836.

Fig. 3. Measured execution time against n on logarithmic axes, with a reference line proportional to n^3 . Excluding the two smallest sizes removes the measurements most distorted by fixed overhead and brings the fitted exponent closer to the theoretical value of 3.

In Fig. 3(a) the measured curve lies almost exactly on the reference line for $n \geq 100$, while the two smallest sizes sit visibly above it. Fig. 3(b) shows the same data with those two points removed, and the measured curve now follows the reference closely across its whole range. Taken together the two panels display both the confirmation of cubic growth at scale and the reason for the depressed exponent when the smallest sizes are included.

VII. COMPARISON WITH A CONVENTIONAL BST

A. Choice of Baselines

The project description requires a comparison against a conventional binary search tree without specifying which construction should be used [1]. Two were implemented, because comparing against only one would be misleading.

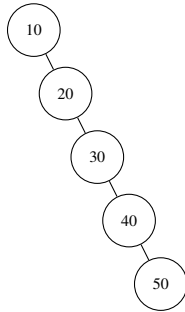
The first inserts the keys one at a time in the order given, using ordinary binary search tree insertion with no balancing. Since the input keys are sorted, every key is larger than all those already present, and the result degenerates into a right chain of height n . This represents the worst case.

The second builds a height-balanced tree by taking the middle key of each range as its root and recursing on the two halves. This ignores the probabilities entirely but produces a tree of minimum possible height, and therefore represents a genuinely well-constructed conventional tree.

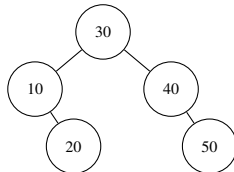
Reporting only the degenerate baseline would overstate the benefit of the dynamic programming approach, since a sorted-order insertion is a known pathological case and outperforming it demonstrates little.

B. Structural Comparison

The two conventional trees over the five-key example are shown in Fig. 4, and may be compared directly against the optimal tree of Fig. 1.



(a) Sequential insertion; cost 3.00, height 5.



(b) Height-balanced construction; cost 1.90, height 3.

Fig. 4. The two conventional binary search trees built over the same five keys

C. Quantitative Comparison

Table XII reports the four measures requested in the project description [1].

TABLE XII
COMPARISON OF THE THREE TREES ON THE SPECIFIED EXAMPLE

Tree	Cost	Avg. depth	Height	Build (s)
Sequential	3.0000	3.0000	5	0.000005
Balanced	1.9000	2.2000	3	0.000004
Optimal	1.8000	2.2000	3	0.000017

The optimal tree reduces the expected search cost by 40.0% relative to the degenerate tree and by 5.26% relative to the balanced tree.

D. Interpretation

The most informative aspect of Table XII is not the improvement over the degenerate tree, which is expected, but the relationship between the balanced tree and the optimal tree.

Both have height 3 and both have an average depth of exactly 2.2. Their depth multisets are identical: each contains one key at depth 1, two at depth 2 and two at depth 3. In structural terms the two trees are equally good, and no purely structural measure distinguishes them.

The entire improvement therefore comes from which key occupies each available depth. Comparing Fig. 1 with Fig. 4(b), the balanced construction places key 10, with probability 0.10, at depth 2 and key 20, with probability 0.20, at depth 3.

The optimal tree exchanges them. The saving is:

$$(0.20 \times 3 + 0.10 \times 2) - (0.20 \times 2 + 0.10 \times 3) = 0.10, \quad (12)$$

which accounts exactly for the difference between 1.90 and 1.80.

This isolates what the dynamic programming algorithm actually contributes. A balanced tree cannot make this exchange, not because its construction is deficient, but because it does not read the probabilities at all.

The improvement is obtained purely from information that the conventional construction discards.

E. Construction Cost

The comparison would be incomplete without noting that the conventional trees are considerably cheaper to build. The balanced construction is $O(n)$ once the keys are sorted, and sequential insertion into sorted input is $O(n^2)$, while the dynamic program is $O(n^3)$.

The optimal tree is therefore worthwhile when the tree is built once and searched many times, and when the access probabilities are known in advance and stable. If the probabilities are unknown, change frequently, or the structure must be rebuilt often, a balanced tree is the more practical choice.

VIII. COMPLEXITY ANALYSIS

A. Time Complexity

The algorithm solves one subproblem for every range $[i, j]$ with $1 \leq i \leq j \leq n$. The number of such ranges is:

$$\binom{n}{2} + n = \frac{n(n+1)}{2} = \Theta(n^2). \quad (13)$$

For each range the algorithm examines every key inside it as a candidate root. A range of length L requires L such evaluations, and L may be as large as n , so each subproblem costs $O(n)$.

Combining the two factors:

$$T(n) = \Theta(n^2) \times O(n) = O(n^3), \quad (14)$$

which agrees with the time efficiency of $\Theta(n^3)$ stated for this algorithm in the course material [2], [3].

The exact operation count follows from summing over all ranges. There are $n - L + 1$ ranges of length L , each requiring L evaluations.

So:

$$\sum_{L=1}^n L(n-L+1) = \frac{n(n+1)(n+2)}{6}. \quad (15)$$

This expression is $\Theta(n^3)$ and is the quantity tabulated in the second column of Table XI. It also confirms that the analysis is tight rather than merely an upper bound: the cubic behavior is attained, not just bounded.

It is worth noting explicitly why the constant-time evaluation of $W(i, j)$ matters to this result. Without the prefix sums of (10), computing the interval weight inside the loop over candidate roots would add a factor of n , giving $O(n^4)$. The measured growth exponent of 2.836 reported in Section VI would then have been closer to 4, and the discrepancy would have been plainly visible in Fig. 3.

The agreement between theory and measurement is therefore also a check on the implementation.

B. Space Complexity

The algorithm stores two tables, each holding one entry for every pair (i, j) . Each table occupies $\Theta(n^2)$ space, and the prefix-sum array adds a further $\Theta(n)$. The reconstructed tree holds n nodes, and the recursion depth of Algorithm 2 is bounded by the height of the tree, which is at most n . The tables dominate, giving:

$$S(n) = \Theta(n^2), \quad (16)$$

again matching the space efficiency in the course material [2].

Since only entries with $i \leq j$ are ever used, slightly less than half of each table is occupied, as Tables VI and IX make plain. A triangular representation would halve the constant factor but would not change the asymptotic order, and the square arrays were retained because they keep the indexing identical to the recurrence.

C. Agreement Between Theory and Measurement

Three separate features of the experimental data support the theoretical analysis. The time per operation in Table XI converges to a constant, which is precisely what a running time proportional to the operation count predicts. The growth ratio between $n = 200$ and $n = 400$ is 8.90 against a predicted 8. The fitted exponent over the larger sizes is 2.836 against a predicted 3, and rises towards 3 as small sizes are excluded.

IX. DISCUSSION

A. Failure of Greedy Root Selection

The intuition that the highest-probability key belongs at the root holds in the five-key example, where key 30 has both the largest probability and the root position, but this is coincidental rather than general.

The ten-key instance of Section VI-E provides a clear counterexample. That input assigns probability 0.18 to key 60, the second largest in the set, and probability 0.08 to key 30. In the optimal tree of Fig. 2, key 60 is placed at depth 3 while key 30, with less than half its probability, is placed at depth 2. A greedy rule that ordered the keys by probability and assigned depths accordingly would produce a different and more expensive tree.

The explanation is that a root choice does not only determine the depth of one key. It partitions the entire range, and the cost of that partition is borne by every remaining key. Placing key 60 higher would require a partition whose total cost exceeds the saving obtained on that single key. This is exactly the interaction between subproblems that a greedy algorithm cannot account for and that dynamic programming resolves by evaluating all candidate roots.

B. Monotonicity of the Root Table

An observable pattern in the computed root tables is that the entries never decrease along a row or down a column. In Table VI the first row reads 1, 2, 3, 3, 3 and the last column reads 3, 3, 3, 4, 5. The larger Table IX makes the pattern still clearer: its first row reads 1, 2, 2, 3, 4, 5, 5, 5, 5, 5 and its last column reads 5, 5, 6, 6, 6, 7, 8, 9, 9, 10, with no decrease anywhere in either direction.

This reflects the property:

$$R[i, j-1] \leq R[i, j] \leq R[i+1, j], \quad (17)$$

which states that extending a range to the right can only move the optimal root to the right, and that removing a key from the left has the same effect.

The course material notes that this monotonicity may be exploited to reduce the time efficiency of the algorithm from $\Theta(n^3)$ to $\Theta(n^2)$, since $R[i, j]$ is always in the range between $R[i, j-1]$ and $R[i+1, j]$ and the search for the optimal root may therefore be restricted to that interval [2]. The standard cubic algorithm was retained as the implementation for this project, in accordance with the project description, and the optimisation is noted here as a known refinement rather than as work carried out.

C. An Anomaly in the Timing Data

The time per operation in Table XI decreases steadily up to $n = 200$ but rises slightly at $n = 400$, from $0.046 \mu\text{s}$ to $0.051 \mu\text{s}$. The most plausible explanation is memory locality. At $n = 400$ the two tables together hold approximately 2.6 megabytes of floating-point and integer data, which exceeds the capacity of the processor's mid-level cache.

A greater proportion of accesses must therefore be served from slower memory, raising the average cost of an operation. This is a hardware effect rather than an algorithmic one, and it does not affect the asymptotic analysis.

D. Extension to Unsuccessful Searches

The formulation used in this project models successful searches only. The course material notes that the method can be expanded to include unsuccessful searches [2], which is achieved by introducing dummy keys representing the intervals between and outside the real keys, so that failed searches may also be assigned probabilities.

The recurrence retains the same shape, with the interval weight extended to include the dummy probabilities. Implementing this variant would be a natural extension of the present work, and would allow the tree to be optimized against a more realistic access model in which some searches fail.

E. Limitations

Three limitations of the present work should be noted. The cubic running time restricts practical use to inputs of moderate size; at $n = 400$ the construction already requires approximately half a second in Python. The probabilities are assumed to be known in advance and to remain fixed, whereas real access patterns often drift over time and would require the tree to be rebuilt. The implementation is written in interpreted Python, so the absolute timings are substantially larger than a compiled implementation would produce, although the growth behaviour is unaffected.

X. CONCLUSION

This project has presented a complete design, implementation and analysis of the Optimal Binary Search Tree problem solved by dynamic programming.

The problem was formulated by defining the state $C[i, j]$ as the minimum expected search cost over a contiguous key range with depths measured relative to that range's own root. The recurrence was derived from the observation that choosing a root pushes every key in the range one level deeper, contributing an additional cost equal to the total probability of the range, and the base case of the empty range was shown to imply the single-key case rather than requiring a separate rule.

The implementation constructs both the cost table and the root table in increasing order of range length, using precomputed prefix sums to evaluate interval weights in constant time and thereby preserving the cubic running time. The optimal tree is reconstructed from the root table, and the resulting expected cost is verified by an independent traversal of that tree.

On the example specified in the project description the algorithm produces an expected search cost of 1.80 comparisons, compared with 1.90 for a height-balanced tree and 3.00 for a tree built by sequential insertion. The comparison against the balanced tree proved the more informative of the two, since both trees share the same height and the same average depth; the entire improvement arises from the placement of keys within an identical structure, using probability information that a conventional construction does not consult. The ten-key instance further showed that this placement cannot be obtained greedily, since the key with the second-largest probability is not placed at the shallowest available depth.

The theoretical complexity of $O(n^3)$ time and $O(n^2)$ space was confirmed experimentally across seven input sizes. The measured cost per inner-loop operation converges to a constant, doubling the input size from 200 to 400 multiplied the running time by 8.9 against a predicted factor of 8, and the fitted growth exponent reaches 2.836 over the larger sizes. Correctness was established independently by a suite of 185 automated tests, by agreement with the worked example published in the course material, and by an exhaustive cross-check against every possible tree shape for inputs of up to seven keys.

REFERENCES

- [1] "Design and implementation of an optimal binary search tree using dynamic programming," Project Assignment, CMP3005 Analysis of Algorithms, Bahçeşehir University, 2026.
- [2] "Chapter 8: Dynamic programming," CMP3005 Analysis of Algorithms lecture slides, Bahçeşehir University, 2026. Based on A. Levitin, *Introduction to the Design & Analysis of Algorithms*, 2nd ed., Pearson Addison-Wesley.
- [3] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 2nd ed. Boston, MA, USA: Pearson Addison-Wesley, 2007.
- [4] W. M. W. Kobahalabi, "optimal-bst-dynamic-programming," (@WalidMWH), GitHub repository, 2026. [Online]. Available: <https://github.com/WalidMWH/optimal-bst-dynamic-programming>